

Appendix I — Comparison Tables

Every row in this appendix's four tables was an option this project weighed, recorded in `docs/Decisions.md` or `docs/Research.md` with a real rationale — not a generic buyer's-guide comparison. Where the project later revisited a choice (the LLM-provider roster moved four times in two weeks), the table shows the sequence, not just the final state, because the sequence is the more instructive artifact.

I.1 Vector Databases

Store	Deployment	Metadata filtering	Verdict
ChromaDB	Local, Python-native, no daemon	Yes — essential for source filtering	Chosen (ADR-002)
FAISS	Local, embedded library	No — needs add-on tooling	Rejected — no metadata support out of the box
Pinecone	Managed cloud	Yes	Rejected — cloud-only; adds latency, cost, and data leaves the machine
Weaviate	Self-hosted, requires a daemon	Yes	Rejected — more operational complexity than a solo local project needs

ChromaDB's local persistence is two components — `chroma.sqlite3` for metadata plus a UUID folder for the HNSW binary index — that must be kept together; losing either breaks the store. Two collections are in active use: `documents` (source chunks) and `learned\_qa` (the self-learning memory covered in Appendix D). Decisions.md's ADR-002 entry gives the verdict in its own words, quoted here complete: “Simplest Python-native API (no daemon). Local persistence (two components: `chroma.sqlite3` for metadata + UUID folder for HNSW binary). HNSW index gives ~95–99% recall at sub-second latency even as corpus grows. Metadata support is essential for source filtering. FAISS rejected: lacks metadata without add-ons. Pinecone rejected: cloud-only, adds latency and cost.”

I.2 Embedding Models

Model	Dimensions	Runs where	Verdict
all-MiniLM-L6-v2	384	Local, CUDA-accelerated	Chosen (ADR-001)
all-mpnet-base-v2	768	Local	Rejected — higher quality, but slower and heavier for this pipeline's latency budget
text-embedding-3-small	1536	OpenAI API (cloud)	Rejected — adds network latency and per-call cost; breaks the fully-local design
bge-small-en-v1.5	384	Local	Considered — similar size class to MiniLM, not adopted

The embedding dimension is fixed project-wide at 384 the moment `all-MiniLM-L6-v2` was chosen: an HNSW index cannot change dimension in place, so switching models later would mean re-ingesting every document. Six transformer layers keep encoding under 100ms per batch on the project's RTX 5050 GPU, and no API call is required — a requirement that ruled out `text-embedding-3-small` on its own, independent of quality, since the project's LLM roles were already juggling enough external-provider dependencies (Appendix N).

I.3 LLM Providers

Unlike the vector store and embedding model, the LLM provider roster was not a single decision — it moved four times in two weeks (ADR-054 → ADR-060 → ADR-061 → ADR-062), and the final configuration was chosen from a real, measured benchmark rather than reasoned about qualitatively. Table I.3 is that benchmark's headline numbers, drawn from `Execution Time Comparison.md` as recorded in Research.md topics 46 and 50 — three queries of increasing complexity, two runs each, per configuration.

Configuration	Avg. time (complex query)	HTTP 402s observed	Outcome
Setup 1 — HF router for judge/fix/CAQ, local for the rest	31:22	≥1 per run, every run	Slowest overall
Setup 2 — Groq for tool-calling, local for everything else	8:36	0	Fastest; recommended pending Setup 5
Setup 3 — HF router for judge/fix/CAQ, Groq for the rest	10:50	Every run	Faster than Setup 1, still HF-dependent
Setup 4 — HF router for judge/fix/CAQ, local for the rest (repeat)	10:38	67–149 per run	Highest 402 density recorded
Setup 5 — all four roles local (custom endpoint)	6:58	0	Adopted (ADR-062) — matches Setup 2 speed, no Groq dependency

Common pitfall — A fast average can hide a broken judge

Setup 4's 10:38 average looks like ordinary network latency until a same-day debug-log audit (Research.md topic 51) checked what the judge calls actually returned: with the Hugging Face router returning HTTP 402 on most requests, `validate\_merge` produced a real, non-fallback verdict only 1 time in 13, `validate\_redundancy` 0 in 9, and `validate\_lbc` 0 in 14 — the rest silently defaulted to `UNKNOWN`/auto-rejected. A latency benchmark alone would never catch this; only cross-referencing the debug log lines behind the timing numbers did.

Setup 5 — every role pointed at one local, OpenAI-compatible custom endpoint — is the configuration shipped today (Appendix N). It won on both axes measured: comparable speed to the best Groq-assisted setup, and zero infrastructure errors across every run, versus every HF-router-dependent setup logging 402s on every single run regardless of what served the remaining roles.

I.4 Tracing / Observability Backends

Definition — Ambient vs. callback-based instrumentation

Ambient instrumentation activates globally — via an environment variable (LangSmith's `LANGCHAIN_TRACING_V2=true`) or by patching SDK classes at import time (Phoenix's OTel auto-instrumentation) — and traces every call automatically, with zero per-call wiring. Callback-based instrumentation (Langfuse) is an explicit object that must be present in a specific call's `config={"callbacks": [...]}` to trace anything, which means it can silently miss calls a multi-node pipeline fails to thread that config through to.

Backend	Self-hosting footprint	Instrumentation model	Strongest at
Arize Phoenix	One container; SQLite (dev) or Postgres (prod)	Ambient (OTel auto-instrumentation)	Evals, dataset/experiment comparison, embedding/retrieval-drift visualization
LangSmith	Self-hosting is an Enterprise add-on; ruled out on cost/complexity	Ambient (env-var activated)	Turnkey tracing UI; no infra to run at all on the managed tier
Langfuse	Postgres + ClickHouse + Redis/Valkey + S3-compatible store — real ongoing operational weight	Callback-based (explicit, must be threaded through config)	Prompt versioning, cost dashboards, Scores/Datasets/Annotation queues

Research.md's own topic-54 conclusion, quoted complete: “For this project specifically (solo dev, no data-egress allowed, no existing SSO/multi-tenant need, and a two-track document/learned_qa retriever whose main open questions are retrieval-quality debugging rather than production cost-attribution), Phoenix was chosen over Langfuse: zero infrastructure weight beyond one container, no feature gating, and its retrieval/eval tooling maps more directly onto the project's actual open bugs (e.g. Bugs.md BUG-031/BUG-035-class retrieval-verdict mismatches) than Langfuse's general-purpose tracing does.” Phoenix was chosen as primary (ADR-063) on this basis. LangSmith is retained in parallel for the same reason the vector-store and embedding-model choices were made early and never revisited without evidence: switching an established, working piece of infrastructure needs a measured reason, not just a newer alternative. Langfuse was added third (ADR-066), not as a replacement for either — its callback-based model required real code changes (explicit `config` threading, ADR-067) that neither Phoenix nor LangSmith ever needed, which is itself the most durable finding of the comparison: it is an architectural difference between the tools, not a version-specific quirk that a future release might close.